Where Did the Agent Go Wrong? Evaluating Failure Attribution Against Counterfactual Rescue

Anonymous ACL submission

Abstract

Self-improving agent loops often begin by asking an LLM which step caused a failure. We ask whether counterfactual rescue is a reliable reference for evaluating such attributions: a method is scored by how much an oracle's correction at the step it names raises the success of continued replays over uncorrected ones. On 300 natural failures of a self-improvement loop in τ^2-bench and AppWorld, a zero-cost rule naming the first state-changing action scored highest under the registered protocol (0.42 vs. 0.20–0.34 for LLM methods; not significant), but three choices of the reference decided the comparison. Without null replays, 46 more failures counted as rescuable through chance recovery and an LLM judge ranked first. Aligned with the metric post hoc, binary search and counterfactual search scored 0.40 and 0.41. And the oracle rescued writes mainly by reading the grading: two independent auditors accepted 3 and 6 of 12 distinct state-changing τ^2 corrections, many of which overrode the user's confirmed choice. A rescue reference therefore needs null controls, objectives matched to the metric and a check that corrections are valid; ours checked validity only in a post hoc audit. A small repair experiment detected no benefit of attribution under the tested configuration.

1 Introduction

Agents that improve themselves from their own failures share a first step. An analyst model reads a failed episode, decides where it went wrong, and hands that diagnosis to a proposer that edits the agent's prompts, tools or code (Shinn et al., 2023; Zhao et al., 2024; Zhang et al., 2026b; Chen et al., 2026; Xia et al., 2026). If the diagnosis points at the wrong step, the proposer may repair the wrong thing. Failure attribution has therefore become a task of its own, with benchmarks of human-labelled multi-agent logs (Zhang et al., 2025; Liu et al., 2026), taxonomies of failure modes (Cemri et al., 2025, 2026; Zhu et al., 2025), and methods that replay a failure with a step changed (Zhang et al., 2026a; Ma et al., 2025; Lin et al., 2026; Bonagiri et al., 2026; Shah, 2026).

Such methods can only be compared against a reference, and for a repair loop the natural one is rescue: a step is to blame if correcting it would have made the episode succeed. Replay-based methods already use this idea to label failures or train attributors. We ask whether counterfactual rescue is a reliable reference for evaluating attribution, and which of its design choices decide what such an evaluation concludes.

Defining the reference was the hard part: the *earliest* step whose single correction beats uncorrected replays was unstable, because a failure usually has several rescuing steps. We therefore measure a *rescue profile*: for every step, the mean gain in success over null replays across four independent corrections by an oracle that reads the task's grading. Three quantities must be kept apart: the reference gain R_k of a step, which belongs to the step and the oracle; a method's score, the gain at the step it names; and whether a repair drafted from an attribution improves the agent. Figure 1 shows a profile whose best rescue overrides a choice the user had confirmed.

We apply the profile to 300 natural failures of agents built by a self-improvement loop, RRSI (Xia et al., 2026), in τ^2-bench's retail and airline domains (Barres et al., 2025) and in AppWorld with a code agent (Trivedi et al., 2024), and compare ten attribution methods, from LLM judges of different cost and design (Zhang et al., 2025) to counterfactual search with up to 40 replays and two structural rules. Methods, metrics and analyses were registered before the main set's rescue profiles were computed. Under the registered protocol a free rule naming the first state-changing action scored highest, but most differences between methods were within sampling noise, and three choices of the ref-

erence decided which method led. We keep the rule as a diagnostic baseline: it shows how much of a rescue score rewards late, committing steps that an oracle with the grading can repair. Our contributions:

- **A null-controlled rescue evaluation whose dependence on the oracle is explicit**: profiles over all steps, null replays that let the agent act again, scores at the named step on a fixed or a re-selected cohort, and a validity-weighted gain that counts an invalid correction as none (§3).
- **Measured effects of three evaluation choices on natural failures** (§5). *Null controls*: without them 46 more failures counted as rescuable and a judge led, while on the registered cohort the rule stayed first. *Objective and information*: the registered judges were asked for the earliest mistake and search lacked the grading; aligned with the metric post hoc, binary search and counterfactual search came within 0.02 of the rule (point estimates; neither shown equivalent). *Intervention validity*: without the grading the oracle rarely rescued writes, two auditors rejected most corrections that overrode the user's confirmed choice, and dropping such corrections moved the rule from first to fifth. Position baselines (the second-to-last step scored 0.34) and scores over all 300 failures (the rule and gain-asked binary search both 0.23) show how much of a lead reflects where a step lies and which failures count.
- **A release** of the failures, profiles, replay outcomes, method outputs, code, and the audit sample with both auditors' labels.

A registered repair experiment in τ^2, too small to detect a useful round's gain and without a positive control, detected no repair benefit of any attribution under the tested configuration; we report it as an exploratory test (§6).

2 Related Work

Failure attribution. Who&When introduced automated attribution of the responsible agent and step in multi-agent logs with all-at-once, step-by-step and binary-search judges, finding steps far harder than agents (Zhang et al., 2025); Who&When Pro revisits its labels (Liu et al., 2026). Taxonomies organise failures into modes (Cemri et al., 2025, 2026; Zhu et al., 2025, 2026b) and ErrorProbe learns diagnosis from its history (Li et al., 2026). MP-Bench argues that a failure often admits several plausible attributions and that earlier conclusions owe much to benchmark design (In et al., 2026). A2P and RAFFLES add counterfactual or verifier reasoning to judges (West et al., 2025; Zhu et al., 2026a); ProcessBench asks for a math solution's first wrong step (Zheng et al., 2025). These evaluations score agreement with labels; we score the effect of correcting the named step, and likewise find that how judges are asked changes their score.

Counterfactual attribution. AgenTracer labels failures by replaying corrected steps of injected faults and trains an attributor on them (Zhang et al., 2026a); DoVer and REFLECT intervene on a trace to validate hypotheses (Ma et al., 2025; Lin et al., 2026); CausalFlow and Causal Agent Replay attribute failures by counterfactual replay (Bonagiri et al., 2026; Shah, 2026). We use this intervention as an evaluation reference, with null replays and a profile over all steps, and show which of its choices move a comparison of methods. The profile is a graded, sampled version of but-for causation (Halpern, 2016; Pearl, 2009).

Self-improving agents. Reflexion, ExpEL and ACE turn failures into lessons or playbooks (Shinn et al., 2023; Zhao et al., 2024; Zhang et al., 2026b); HarnessFix diagnoses and repairs harness flaws (Chen et al., 2026); RRSI digests failed traces, proposes harness edits and keeps them by re-evaluation (Xia et al., 2026). Prediction of failure does not imply that intervening helps (Vasudev et al., 2026). We evaluate the diagnostic step these loops share, on failures one of them produced.

3 A Reference: Rescue Profiles

Replays. A failed episode is a sequence of agent steps $0, \ldots, n-1$. A *replay from step* k keeps steps $0, \ldots, k-1$ and the environment state they produced, and lets the same agent and user simulator continue; a *corrected* replay forces a given action at step k first. The agent runs at temperature 0, but replays still vary, so each outcome is a sample.

Oracle corrections. For each step an oracle (DeepSeek-V4-Pro, thinking off) sees the domain policy, the tools, the episode up to the step and the task's grading criteria: τ^2's gold actions and required information, or AppWorld's unit tests with

their expected values. It either judges the step correct or writes the action the agent should have taken, using only what the agent knew at that point: the criteria tell it the right outcome, not identifiers or facts the agent had not yet seen. The reference is thus repairability by an oracle that knows the intended outcome, privileged information an offline evaluation may use; it does not certify that each correction was justified by what the agent had seen.

Rescue gain. This protocol replaced an earliest-step protocol whose independent runs agreed on 42% of pilot pairs (Appendix A). At every step k we draw $K = 4$ oracle samples. Each sample that gives an applicable correction gets 2 corrected replays, and a step with at least one such sample gets 4 null replays (54%, 60% and 39% of steps in retail, airline and AppWorld; other steps have $R_k = 0$). The rescue gain of step k is

$$R_k = \frac{1}{K} \sum_{i=1}^{K} c_{k,i} \big(\hat{p}_{k,i}^{\text{corr}} - \hat{p}_k^{\text{null}} \big), \qquad (1)$$

where $c_{k,i} = 1$ if sample i gives an action the replay can apply, and $c_{k,i} = 0$ if it judges the step correct or its action cannot be applied. R_k is thus the share of samples that propose a correction times the mean gain of those corrections: how useful pointing at step k is to this correction procedure, not a property of the step alone, since a low R_k can mean that the oracle found no correction (Appendix Table 10 separates the two). A null replay lets the agent act again at step k; it repeated the observed tool calls in 91–95% of τ^2 null replays but the whole turn, message included, in only 35–36%, and the observed code in 58% of AppWorld ones (post hoc). R_k thus compares the oracle's action with letting the agent act again, not with keeping its action (Appendix Table 7 uses the observed action as control). A failure is rescuable if $\max_k R_k \geq 0.5$, and then its best rescue step is the step with the largest gain (earliest on ties). We also record the earliest step with $R_k \geq 0.5$. Figure 1 shows one profile.

Scoring a method. A method names one step $\hat{k}$. Its score on a failure is $R(\hat{k})$, the rescue gain at that step (negative gains count), and 0 for an invalid step. The primary metric is the mean of $R(\hat{k})$ over rescuable failures; it rewards naming any step whose correction helps, in proportion to how much it helps. Secondary metrics include exact agreement with the best rescue step (Appendix Table 5).

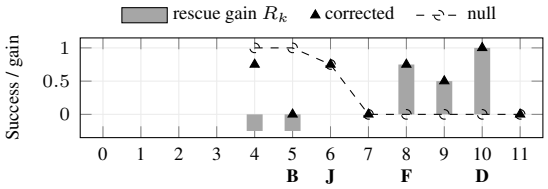


Figure 1: Rescue profile of one retail failure (nulls only where some sample corrected). The agent proposed the silver one of two 4K waterproof cameras and the user accepted; the task names no colour, but the grading expects the cheaper black one. At step 6 (J: all-at-once judges) corrected and null replays both succeeded 3 times in 4. At steps 8–10 (F: first write; D: best rescue step) the oracle swapped in the black camera, overriding the user's confirmed choice; both auditors rejected the same override in another failure of this task. B: binary search.

Valid rescue. An oracle that reads the grading can rescue a failure with an action the agent could not have justified, such as overriding a choice the user had confirmed (Figure 1). With a label $v_{k,i} \in \{0, 1\}$ for whether correction i at step k is valid, the valid rescue gain

$$R_k^{\text{valid}} = \frac{1}{K} \sum_{i=1}^{K} c_{k,i}\, v_{k,i} \big(\hat{p}_{k,i}^{\text{corr}} - \hat{p}_k^{\text{null}} \big) \qquad (2)$$

keeps K as the denominator, so an invalid correction counts as no correction. The filters of §5 set v mechanically and the audit estimates it on a sample; a method's valid score would need labels for every correction at the steps it names, which we did not collect.

4 Setup

Failures. All failures come from runs of RRSI (Xia et al., 2026) as a self-improvement loop on a DeepSeek-V4-Flash agent (one per domain, and two further retail runs for 14 failures): from the harnesses accepted over 20 rounds, in their evaluations on training tasks and in their deployment on held-out tasks. In τ^2-bench (Barres et al., 2025) the agent serves a simulated customer through tool calls; in AppWorld (Trivedi et al., 2024) it writes Python against the APIs of nine apps and submits with a final call. We drew 100 failed episodes per domain with a registered rule: no harness error, at most two per task and harness, tasks taken round-robin in a seeded order. They cover 36 (retail), 24

(airline) and 47 (AppWorld) tasks and average 11.7, 10.3 and 19.6 agent steps.

Methods. Every LLM method except counterfactual search reads the failure as RRSI's analyst reads it: the task (in τ^2 with the simulated customer's hidden instructions), the episode with numbered steps, and the grading report the agent never sees, which lists τ^2's gold actions and missing information or AppWorld's failed unit tests with their assertions. The judges therefore have the information the oracle has. Each method names one step.

- **Rules**: the *first write*, the first step that calls a state-changing tool or API (else the last step), and the *last step*.
- **Judges** (Zhang et al., 2025): *all-at-once*, one call over the whole episode, with DeepSeek-V4-Flash, V4-Pro, and V4-Pro with thinking; *step-by-step*, one call per step, in order, until a step is judged decisive; *binary search*, halving the episode $\lceil \log_2 n \rceil$ times and asking which half holds the decisive mistake.
- **Attributors**: a *root-cause attributor*, one call asking for a root cause, the earliest wrong step and a component; and RRSI's own multi-turn *trace digester* (Xia et al., 2026), of which we take the earliest step it cites as evidence.
- **Counterfactual search** (CF search), replay-based like AgenTracer's labelling (Zhang et al., 2026a) but without grading criteria: one call names up to five suspect steps with corrected actions; suspects are tested in order with 4 corrected and 4 null replays, and the search stops at the first correction that beats null by 2 successes or when its budget of 40 replays runs out; otherwise it answers with the first suspect.

Judges are prompted for the decisive mistake, defined as the earliest step whose correction would most likely have rescued the episode (Appendix B); post hoc reruns change this definition or withhold the grading information. Pro runs with thinking off unless stated.

Registration and analysis. Methods, prompts, metrics and analyses were registered before the main profiles were computed (AppWorld's methods ran alongside its profiles), and no method sees a profile. The registered primary comparison is counterfactual search at budget 40 against the all-at-once judge with Pro: the paired difference in $R(\hat{k})$ over rescuable failures, with a 95% bootstrap interval over task clusters (10,000 draws) and a paired permutation test flipping signs by task, on retail and airline pooled and, for AppWorld, separately. Everything else is estimated with cluster-bootstrap intervals and no further tests; unregistered analyses are marked post hoc or exploratory, and their intervals are not adjusted for multiplicity. Intervals resample tasks and hold each failure's profile fixed; post hoc, redrawing each step's oracle samples and replays as well widened them only slightly (the rule's lead over binary search: $[-0.03, +0.18]$ instead of $[-0.01, +0.18]$ in the same draws; Appendix F). A seeded 10% of failures in each domain got a second, independent profile. Appendix G lists every change made after registration.

5 Results

Profiles. Of 300 failures, 157 were rescuable: 65 of 100 in retail, 37 in airline and 55 in AppWorld. Null replays from the same step succeeded 27%, 18% and 11% of the time; without them, many rescues would be chance recoveries. A third of the rescuable failures (18, 11 and 26) had two or more steps with a gain of at least 0.5. In the retest, 27 of 30 failures got the same verdict on rescuability and 15 of the 17 that both found rescuable the same best rescue step (Appendix A).

Under the registered prompts, the first-write rule scored highest. Pooled over the three domains, the first write scored 0.42 [0.33, 0.51] and the LLM methods 0.20 (step-by-step) to 0.34 (binary search; Table 1); the lead over binary search was not significant ($+0.08$ $[-0.02, +0.18]$, exploratory). Over all 300 failures, a cohort chosen without the reference, the first write also scored highest among registered methods (0.23 against 0.11–0.19), and exact agreement with the best rescue step ordered the methods nearly the same way (Appendix Table 5). The oracle proposed corrections about equally often at every method's steps (0.52–0.64 of samples), but its corrections worked better at the first write's (mean gain 0.61, against 0.34–0.51; Appendix Table 10). Cost bought little (Appendix Figure 3): the all-at-once judges with Flash, Pro and Pro with thinking scored 0.30, 0.30 and 0.31 at \$0.001 to \$0.014 per failure, and on Who&When our judges reproduce the published pattern (Appendix D). Post hoc, a random step scored 0.11 and the step at the first write's mean position (78% through) 0.17, but the second-to-

Method	Model	Mean rescue gain $R(\hat{k})$, rescuable failures [95%] Retail (65)	Airline (37)	AppWorld (55)	Pooled (157)	All failures (300)	$ per failure
First write	rule	0.53 [0.38, 0.65]	0.27 [0.13, 0.45]	0.39 [0.26, 0.52]	0.42 [0.33, 0.51]	0.23 [0.17, 0.30]	0
Last step	rule	0.11 [0.02, 0.24]	0.13 [0.04, 0.23]	0.36 [0.23, 0.50]	0.20 [0.13, 0.29]	0.11 [0.07, 0.16]	0
All-at-once	Flash	0.29 [0.19, 0.38]	0.41 [0.28, 0.53]	0.25 [0.14, 0.37]	0.30 [0.24, 0.37]	0.19 [0.14, 0.23]	0.0010
All-at-once	Pro	0.30 [0.19, 0.41]	0.34 [0.21, 0.48]	0.26 [0.15, 0.38]	0.30 [0.23, 0.36]	0.17 [0.13, 0.22]	0.0031
All-at-once	Pro, thinking	0.26 [0.17, 0.35]	0.42 [0.28, 0.57]	0.28 [0.18, 0.40]	0.31 [0.24, 0.38]	0.18 [0.14, 0.23]	0.0141
Step-by-step	Pro	0.14 [0.03, 0.25]	0.16 [0.04, 0.25]	0.29 [0.18, 0.40]	0.20 [0.13, 0.26]	0.11 [0.07, 0.15]	0.0099
Binary search	Pro	0.33 [0.21, 0.47]	0.34 [0.21, 0.46]	0.35 [0.24, 0.45]	0.34 [0.27, 0.41]	0.19 [0.15, 0.24]	0.0064
Root-cause attributor	Pro	0.26 [0.16, 0.38]	0.36 [0.21, 0.53]	0.22 [0.10, 0.34]	0.27 [0.20, 0.34]	0.16 [0.12, 0.21]	0.0031
RRSI trace digester	Pro	0.17 [0.07, 0.29]	0.35 [0.24, 0.49]	0.19 [0.11, 0.27]	0.22 [0.16, 0.29]	0.14 [0.10, 0.17]	0.0077
Counterfactual search@40	Pro, replays	0.22 [0.12, 0.33]	0.31 [0.19, 0.44]	0.28 [0.17, 0.41]	0.26 [0.20, 0.34]	0.15 [0.11, 0.19]	0.0222
Post hoc variants							
Random step	chance	0.10 [0.08, 0.12]	0.11 [0.08, 0.14]	0.12 [0.09, 0.14]	0.11 [0.09, 0.12]	0.06 [0.05, 0.07]	0
Second-to-last step	rule	0.44 [0.27, 0.58]	0.25 [0.14, 0.39]	0.27 [0.16, 0.39]	0.34 [0.25, 0.42]	0.19 [0.13, 0.25]	0
All-at-once, gain	Pro	0.39 [0.28, 0.51]	0.42 [0.28, 0.55]	0.29 [0.18, 0.42]	0.36 [0.29, 0.44]	0.21 [0.16, 0.26]	0.0031
Binary search, gain	Pro	0.44 [0.33, 0.55]	0.42 [0.25, 0.60]	0.35 [0.23, 0.47]	0.40 [0.33, 0.48]	0.23 [0.18, 0.29]	0.0071
All-at-once, no grading	Pro	0.17 [0.08, 0.26]	0.32 [0.19, 0.45]	0.25 [0.14, 0.35]	0.23 [0.17, 0.30]	0.13 [0.09, 0.17]	0.0028
Binary search, no grading	Pro	0.21 [0.10, 0.33]	0.24 [0.12, 0.35]	0.24 [0.14, 0.35]	0.23 [0.16, 0.29]	0.13 [0.09, 0.17]	0.0064
CF search@0	Pro	0.22 [0.14, 0.32]	0.13 [0.02, 0.24]	0.28 [0.17, 0.40]	0.22 [0.16, 0.28]	0.12 [0.08, 0.16]	0.0037
CF search@0, grading	Pro	0.26 [0.17, 0.37]	0.24 [0.13, 0.34]	0.31 [0.18, 0.44]	0.27 [0.21, 0.34]	–	0.0037
CF search@40, grading	Pro, replays	0.34 [0.25, 0.45]	0.38 [0.26, 0.52]	0.34 [0.22, 0.46]	0.35 [0.28, 0.42]	–	0.0152
CF search@40, aligned	Pro, replays	0.44 [0.35, 0.53]	0.48 [0.35, 0.62]	0.32 [0.21, 0.43]	0.41 [0.34, 0.47]	–	0.0275

Table 1: Attribution scored by the rescue gain at the named step. Intervals: cluster bootstrap over tasks, with profiles held fixed (§4). All failures: mean over all 300, rescuable or not, a cohort chosen without the reference (–: run on the rescuable failures only). Cost: mean over domains of list-price dollars per failure (tokens and replays); most judges cost 2 to 4 times as much in AppWorld as in τ^2. Lower block, post hoc (§5): the expected score of a random step and the second-to-last step; judges asked for the largest gain over null (gain) or without grading information; search's first suspect before any replay (@0) and search given the grading information.

last step 0.34, level with binary search (retail 0.44, above every registered LLM method); the first write led it by 0.08 $[+0.02, +0.15]$ and ranked first among the registered methods in 94% of task-clustered bootstrap samples. Scored against the mean gain of steps in the same position decile, the second-to-last step fell to 0.10 while the first write still led (0.24 against 0.19 for binary search, $+0.05$ $[-0.04, +0.14]$; Appendix F).

Asked for the measured quantity, judges came close to the rule. The registered prompts asked for the earliest decisive mistake, while R_k measures the gain of a correction over letting the agent act again, and judges that missed the best rescue step usually named an earlier one. Post hoc, asked for the step whose correction would most increase success over letting the agent act again (Appendix B), the Pro judges rose by 0.07 (all-at-once 0.30 to 0.36, $[+0.02, +0.12]$; binary search 0.34 to 0.40, $[-0.01, +0.13]$) and the Flash judge by 0.01; the first write then led binary search by 0.02 ($[-0.07, +0.10]$, too wide to show equivalence) and ranked first in 63% of bootstrap samples (binary search in 33%). Over all 300 failures the two scored 0.23 each, and with failures selected on half of each step's oracle samples and null replays and scored on the other half, binary search asked for the gain came within 0.01 of the rule ($[-0.06, +0.08]$; Appendix F).

Aligned with the judges and the metric, search came close to the rule. Counterfactual search at budget 40 scored 0.26, at seven times the cost of the all-at-once judge with Pro ($0.022 against $0.003 per failure); the registered comparison with that judge detected no difference (-0.061 $[-0.146, +0.032]$ in τ^2, $p = .20$, $n = 102$; $+0.025$ $[-0.087, +0.136]$ in AppWorld, $p = .69$, $n = 55$). Unlike the judges, search did not read the grading, asked where the agent went wrong, and stopped at the first suspect passing a noisy threshold. Post hoc, with all three differences removed (grading information and replays each added 0.06–0.07), search scored 0.41 (Table 1), $+0.15$ $[+0.07, +0.22]$ above its registered form and 0.01 below the first write in point estimate ($[-0.09, +0.10]$; the rule ranked first in half of

bootstrap samples; Appendix Table 19). It ran only on the 157 rescuable failures, so it has no score over all 300. Aligned search tests corrections it writes with the grading against null replays, much as the reference does, so its score partly reflects agreement with the reference's own procedure.

The rule's lead depends on the kind of step. The first write led in retail (0.53, against 0.14–0.33 for LLM methods) and AppWorld (0.39); in airline the all-at-once judges with Flash and with thinking led (0.41–0.42, against 0.27). Best rescue steps were mostly writes in retail (36 of 65) and utterances in airline (25 of 37). Post hoc (Appendix Table 15), the first write scored 0.60 on the 82 failures best rescued at a write or AppWorld submission, against 0.46 for binary search asked for the gain ($+0.13$ $[+0.03, +0.23]$), and 0.23 on the other 75, against 0.34–0.39 for the best judges and informed search; with the eight domain-by-kind cells weighted equally, the two scored 0.41 and 0.43.

Null replays decided which failures count. Under most other choices the first write stayed first (Kendall's τ of 0.78 to 1.00 against the registered ordering, within the spread of bootstrap samples of the failures, median 0.82; Appendix Table 7): fewer samples, other thresholds, single-rescue failures, split samples, a second oracle model (V4-Flash), the observed action as control, or without the 2.5% of corrections a mechanical screen flagged for an unseen identifier or extra calls (Appendix Table 12). Table 2 shows the choices that moved it. Without null replays 203 failures were rescuable and the all-at-once judge with thinking ranked first, but on the registered 157 the first write stayed first (0.46 against 0.44, $\tau = .87$). Uncorrected replays from an episode's first fifth succeeded 41–44% of the time and from its last fifth 2–6% (Appendix Figure 2), so without the control the evaluation admits failures that recover anyway; LLM methods, which named earlier steps (53–70% of the way through, against 77% for best rescue steps), gained most (0.08–0.13, against 0.04). Even with the control, 91 of the 157 rescuable failures had a step from which uncorrected replays succeeded at least half the time; on the other 66 the first write still ranked first (0.41 against 0.38, $\tau = .87$; post hoc).

Without the grading, the oracle rarely rescued writes. Post hoc, we reran the oracle without the grading, telling it only that the episode failed, at the

	Re-selected			Fixed 157†	
Evaluation choice	Resc.	Rank	FW − best [95%]	Rank	FW − best [95%]
Registered	157	1	$+.08$ $[-.02, +.18]$		
Judges, search aligned*	157	1	$+.01$ $[-.09, +.10]$		
No null replays	203	6	$-.05$ $[-.14, +.05]$	1	$+.02$ $[-.10, +.12]$
Oracle without grading				7	$-.06$ $[-.10, -.02]$
Confirmed choices changed‡	135	5	$-.04$ $[-.13, +.06]$	5	$-.04$ $[-.12, +.05]$
Replaced writes‡	126	8	$-.08$ $[-.18, +.03]$	7	$-.08$ $[-.16, +.01]$
All writes‡	81	8	$-.16$ $[-.27, -.04]$	8	$-.11$ $[-.18, -.05]$

Table 2: The first write (FW) under each evaluation choice (post hoc except the first row): rank among the ten methods of Table 1 and difference from the best LLM method, on failures rescuable under the choice and on the registered 157 (†). Best LLM method: binary search (registered), counterfactual search (no grading; *: aligned search), else the all-at-once judge with thinking. The oracle without grading ran only at steps some method named, so only the fixed cohort is scored. ‡Dropped after the audit: τ^2 corrections changing a choice the user had just confirmed, those replacing the agent's write, every write (which removes the first write's own corrections, so its rank falls by construction).

960 steps any method named on the 157 rescuable failures ($4.0; Appendix Table 16). Gains at best rescue steps fell from 0.76 to 0.17. Corrections of writes lost nearly everything (retail 0.77 to 0.00, AppWorld 0.82 to 0.20), corrections of utterances half to two thirds (0.72–0.76 to 0.24–0.36): the oracle repairs writes mainly by knowing which write the task wanted. Under this oracle every method scored 0.03 to 0.11 and the first write ranked seventh (0.05; Table 2). Its full profiles on a random 100 failures ($3.6; Appendix Table 18) found 17 rescuable, against 51, none in τ^2 best rescued at a write. Removing the grading also weakens the oracle, so this is a sensitivity analysis, not an admissibility test.

Auditors rejected many state-changing corrections. Two auditors independently labelled a stratified sample of 60 corrections with a positive gain (52 distinct; post hoc; Appendix D). They judged 40 and 44 valid given what the agent could see and the policy (80% agreement, $\kappa = .53$), and 3 and 6 of the 12 distinct τ^2 corrections that change state, under a lenient reading adopted during the audit that judged a correction's content (29 and 34 valid if every label given under it is counted invalid). Both rejected most corrections that changed the item, option or reason the user had last confirmed (7 and 5 of 7; Figure 1). We dropped such corrections in two ways, written after the audit (Table 2). A direct filter drops τ^2 corrections that change the agent's write and lose a value the user

had just stated or been shown (236 of 3,103): 135 failures stayed rescuable and the first write ranked fifth (0.30, against 0.34 for the all-at-once judge with thinking). Dropping every replacement of the agent's write (330), which removes 63% of the τ^2 corrections at the first write's steps (the direct filter 49%), left 126 and the first write eighth (0.26). Neither gap was significant, but the rule's lead rests on such overrides.

6 An Exploratory Test of Attribution in Harness Repair

Design. A rescue score is a proxy: a loop needs attributions only to repair its agent. In a separately registered experiment we re-ran one unmodified RRSI round at eight incumbent states, the harnesses that two other runs of the loop in each τ^2 domain had accepted by rounds 5 and 15, changing only the proposer's evidence: **no attribution** (a note asking it to read the traces itself); **RRSI analysis** (the round's own report and trace digests); **first write**, **binary search** (Pro) or **counterfactual search@40** (for each failing trace, the named step and an excerpt of it, with no reason or method name; Appendix B); post hoc, the **oracle step** (the step of largest rescue gain, for the 60 traces where some step had a positive gain); and two post hoc **no-reading** groups whose proposer cannot open failing traces, given RRSI's analysis or the oracle's step, diagnosis and most successful correction (*oracle fix*). The proposer works from the 81 failing traces of the incumbent's evaluation on training tasks and, except in the no-reading groups, can read each, grading report included. Each round drafts two candidates and puts them through RRSI's critic, smoke test and full evaluation, and RRSI's rule decides whether to accept one. A candidate is *deployable* if it passes these gates with a harness error rate of at most 2%; every deployable candidate and every incumbent was deployed on 240 held-out episodes.

Analysis. A slot's gain G is its held-out success minus its incumbent's, in points, and 0 if it was not deployable, since the loop then keeps the incumbent. A contrast is the mean over the eight states of the difference in two groups' mean gains. The six primary contrasts compare each step attribution with no attribution and with RRSI's analysis, with p values from permuting groups within states (Holm-adjusted) and 95% intervals from a t distribution over states; a 90% interval within ± 2.5 points counts as equivalent. The design (standard error 0.9 points per contrast, a minimum detectable difference of 2.6 to 3.2 points) was registered before any attribution was run on these traces, and held-out results stayed blind until all deployments had finished. A useful round gains under a point, so only large effects were detectable.

No detectable repair benefit under the tested configuration. Of the five registered groups' 80 candidate slots, 68 produced a draft and 54 a deployable candidate (Table 3), and no primary contrast was significant (Holm-adjusted $p = 1.0$). Repairs drafted from first-write and binary-search steps were within the margin of those drafted from RRSI's analysis ($+0.31$ $[-0.56, +1.18]$ and $+0.29$ $[-0.82, +1.39]$ points; their 90% intervals exclude differences beyond 1.0 and 1.2), and counterfactual search was 2.14 points below it $[-5.67, +1.40]$, mostly through two airline candidates. The contrasts with no attribution are dominated by one of its candidates, whose write gate blocked nearly every write (-72.5). Post hoc (\$5.9 and \$12.4), the oracle pointer (-0.05 $[-1.97, +1.87]$) and the oracle fix ($+0.39$ $[-0.98, +1.76]$) did no better than RRSI's analysis, blocking the reads changed nothing (-0.05 $[-1.31, +1.21]$), and dropping the 12 slots without a draft changed no conclusion (Appendix F).

Why the test was weak. Post hoc, only 33 of the 81 failing traces were rescuable (\$5.0); on them the groups' pointers scored 0.26 (RRSI's digests) to 0.41 (binary search) and correlated only weakly with candidates' gains (Spearman .15 within states, $p = .14$). On fresh replays (\$2.3; Appendix Table 17) the oracle pointers' gain exceeded binary search's by only 0.06 $[-0.01, +0.14]$ over all 81 traces, and 15 of the 60 oracle fixes replaced the agent's write, 6 of them overriding a choice the user had just confirmed. Rounds rarely accepted a candidate (11 of 64; Appendix Table 14). Proposers opened 97–100% of failing traces, and drafts given the oracle's evidence listed more of its traces among the tasks they expected to fix, yet those tasks improved no more than others (Appendix F). Without a positive control, the experiment cannot show whether attribution limits this loop.

7 Discussion

What the score measures. R_k belongs to a step and an oracle: a low score can mean the oracle found no correction, and a high one marks where

Evidence for the proposer	$R(\hat{k})$ of pointers retail / airline	Deployable	Mean G, all 16 slots both	retail	airline	Deployable mean G	$G > 0$	RRSI accepted	Round level
No attribution	–	10/16	−4.38	−9.27	+0.52	−7.00	50%	2	−0.10
RRSI analysis	0.30 / 0.21	13/16	−0.34	−0.31	−0.36	−0.42	38%	1	−0.10
First write	0.37 / 0.16	10/16	−0.03	+0.21	−0.26	−0.04	60%	2	+0.47
Binary search	0.32 / 0.51	9/16	−0.05	+0.16	−0.26	−0.09	44%	0	0.00
Counterfactual search@40	0.27 / 0.32	12/16	−2.47	−0.26	−4.69	−3.30	25%	2	+0.16
Oracle step	0.67 / 0.81	10/16	−0.39	−1.04	+0.26	−0.62	50%	1	+0.26
RRSI analysis, no reading	0.30 / 0.21	14/16	−0.39	−0.42	−0.36	−0.45	21%	2	−0.05
Oracle fix, no reading	0.67 / 0.81	10/16	0.00	+0.42	−0.42	0.00	60%	1	+0.10

Primary contrast	Difference [95%]	p	Holm	Equiv. (± 2.5)	Round level [95%]
First write − none	+4.35 [−5.97, +14.67]	0.52	1.00	no	+0.57 [−0.19, +1.34]
Binary search − none	+4.32 [−6.90, +15.55]	0.66	1.00	no	+0.10 [−0.14, +0.35]
CF search@40 − none	+1.90 [−10.24, +14.04]	0.91	1.00	no	+0.26 [−0.99, +1.51]
First write − RRSI	+0.31 [−0.56, +1.18]	0.65	1.00	yes	+0.57 [−0.32, +1.46]
Binary search − RRSI	+0.29 [−0.82, +1.39]	0.60	1.00	yes	+0.10 [−0.14, +0.35]
CF search@40 − RRSI	−2.14 [−5.67, +1.40]	0.31	1.00	no	+0.26 [−0.99, +1.51]
Oracle − RRSI	−0.05 [−1.97, +1.87]	0.98	–	yes	+0.36 [−0.27, +0.99]
Oracle − step groups	+0.46 [−1.62, +2.54]	0.75	–	yes	–
Oracle fix − RRSI, no reading	+0.39 [−0.98, +1.76]	0.53	–	yes	+0.16 [−1.44, +1.76]
RRSI: no reading − reading	−0.05 [−1.31, +1.21]	0.97	–	yes	+0.05 [−1.55, +1.66]
Oracle fix, no reading − oracle step	+0.39 [−0.78, +1.56]	0.61	–	yes	−0.16 [−0.85, +0.54]

Table 3: Repairs drafted from each kind of evidence in τ^2: 8 states × 2 candidates per group. G: held-out success of a candidate minus its incumbent's, in points, 0 if not deployable. $R(\hat{k})$: rescue gain of the steps a group pointed to, on the 33 rescuable failures among the experiment's 81 failing traces (post hoc; for RRSI, the earliest step its digests cite). Accepted: by RRSI's own rule. Round level: gain of the accepted candidate, 0 if none. Contrasts: mean over states of group differences, p from within-state permutation; Equiv.: 90% interval within ± 2.5 points, the registered margin, which exceeds a useful round's gain. The oracle and no-reading groups and their contrasts are post hoc and not Holm-adjusted.

an oracle that knows the intended outcome can recover the failure, not where it began. Uncorrected replays rarely succeed late in an episode, so late steps have more room for gain, which favours a rule naming late, committing steps (the first write lies 78% of the way through, and the second-to-last step scored 0.34). In τ^2 a rule that reads the gold actions and names the agent's first departure from them scored 0.50, above the first write's 0.44 (post hoc; Appendix Table 9): there the reference largely rewards finding where the agent left the graded path. The first write is therefore best read as a diagnostic baseline: its score shows how much of a rescue reference rewards late, committing steps that an oracle with the grading can repair.

Implications for self-improving loops. No evidence brought a detectable benefit, and most edits moved held-out success by less than 5 points (Appendix Figure 4); without a positive control, a bottleneck after attribution remains a hypothesis. RRSI's re-evaluation rejected every edit that misfired broadly, protecting the agent more than the attribution behind its repairs.

8 Conclusion

Counterfactual rescue can serve as a reference for failure attribution only with safeguards. On 300 natural failures most methods could not be told apart, and which ranked first depended on three choices: null replays decided which failures count, the objective and information given to the methods decided how close they came to a free rule, and an oracle that reads the grading rescued writes with corrections that auditors often rejected. Rescue-based benchmarks should measure gains against null replays on a fixed cohort, report position baselines and scores over all failures, ask methods for what the metric scores, and fix validity rules in advance, have corrections labelled independently and score the valid rescue gain. A small repair experiment detected no benefit of attribution under the tested configuration.

Limitations

Models and data. All agents, oracles and attributors use one model family (DeepSeek-V4); the second oracle is a smaller model of the same family, and we had API access to one provider only, so no oracle from another family was run. The oracle and the Pro judges are the same model, but the oracle proposed corrections about as often at every method's steps (0.52–0.64 of samples). The failures come from one self-improvement loop and three domains, with 37 to 65 rescuable failures each, so per-domain intervals are wide. We did not label why each failure occurred (agent, user simulator or task specification): the task in Figure 1 names no colour while its grading expects one, and 91 of the 157 rescuable failures had a step from which uncorrected replays succeeded at least half the time. Fourteen retail failures were drawn after the others, from the two further runs of the loop that the repair experiment also used (one trace is in both). Two of 6,540 AppWorld replays could not run (Appendix G). Who&When is a multi-agent benchmark with a different format; only methods without replay were run there.

The reference. It rests on one kind of intervention, a single action corrected by an oracle that sees the grading: it says where a correction informed by the intended outcome helps, not where an error began, and late steps, where uncorrected replays rarely succeed, have more room for gain. Without the grading the oracle rarely rescued writes and the ranking changed; its full profiles cover a random 100 failures, too few rescuable ones for a stable ranking. An oracle told the task's goal in words, without gold actions or tests, would lie between the two we ran; we did not run one. A mechanical screen found few corrections that use unseen values or do more than one step, but it cannot detect a choice made with the grading. Individual best rescue steps are noisy: split halves agreed on 56%, so per-failure labels need gains estimated on held-out samples. The threshold of 0.5 and four samples per step were fixed on a 20-failure pilot.

The audit. Two auditors labelled a stratified sample of 60 corrections without seeing each other's labels (Appendix D); it holds 52 distinct corrections, only 9 at the first write's steps. Agreement was moderate on validity (80%, $\kappa = .53$) and scope ($\kappa = .40$) and low on unseen information ($\kappa = .20$; each found 4 cases, one in common). During the audit both adopted a lenient reading, judging validity by a correction's content and several calls of one kind in one turn as one step (11 and 12 labels, marked in the release, most of them airline items). Counting every label given under it as invalid, 29 and 34 corrections were valid ($\kappa = .64$), so the audit brackets validity rather than measuring it under one fixed reading, and it does not check the screen's low rates of unseen information and extra steps. The sample is stratified, so its shares are not population estimates; of the 41 corrections without a mechanical flag, 28 and 30 were valid. Our filters were written after the audit, so their match with the auditors is in sample (the direct filter agrees with them on 7 and 8 of the 12 distinct writes); the direct filter misses some overrides and the broader ones also drop valid corrections. A stronger test would fix admissibility rules before scoring, have every correction at the compared methods' steps on fresh failures labelled independently under written criteria with adjudication, and compare the aligned methods by valid rescue gain (§3). We release failures, profiles, replay outcomes and code, and the audit sample with both auditors' labels (without free-text notes) and the mechanical flags.

Post hoc analyses. Many analyses are post hoc, with intervals not adjusted for multiplicity: the robustness checks, the observed-action control, the second oracle, the oracle without grading, the split by kind of step, the admissibility screen, the human audit, the filters of state-changing corrections and the fixed-cohort rankings, the failures that recover by chance, the random and position baselines, position-corrected scores and bootstrap ranks, the scores over all 300 failures, the independent-halves scores and the two-level bootstrap, the prompt variants, the judges without grading information, the informed search, search with every suspect tested, the full profiles without grading, the gold-deviation rule, the reading logs and the drafts' targets, and in the repair experiment the rescue scores of its pointers and their fresh replays, its run-grouped and accepted-round intervals, its oracle and no-reading groups, the screen of the oracle fixes and the contrasts without the slots that drafted nothing. Our registered prompts asked for the earliest decisive mistake while the metric rewards the largest gain over null, and registered search stopped at the first passing suspect, so the registered comparison understates the judges and search; the aligned configurations were found on the failures they are scored

on, and confirming them needs a fresh cohort with the configurations fixed in advance (paired intervals were about ± 0.1 with our 157 rescuable failures, and half as many would give about ± 0.14, so a confirmation needs a cohort as large as ours). Our counterfactual search labels steps by replay but trains no attributor, unlike AgenTracer (Zhang et al., 2026a), so its results do not speak to trained attributors.

The repair experiment. It covers τ^2 only, with eight states from four runs of the loop, 16 candidate slots per group and 33 rescuable failures among its 81 failing traces. Its margin of 2.5 points reflects the noise of one deployment and exceeds a typical round's gain (the source runs moved held-out success by -4.3 to $+7.9$ over 20 rounds), and its intervals widen when states are grouped by run; it rules out large differences from RRSI's analysis, not small ones that would compound over a long loop. Only 11 of 64 rounds accepted a candidate, undeployable slots count as no gain, and its contrasts with no attribution are dominated by one candidate. It has no positive control: we did not show that this proposer turns useful evidence into held-out gains at these states, so its null result cannot separate weak attribution from an insensitive repair process. A positive control would remove from an incumbent a component known to matter, such as a policy hint whose removal costs training success, and compare rounds given no attribution and a validated diagnosis naming that component and a failing step. The manipulation was weak: proposers that could read also saw each trace's grading report, so a step pointer added little they could not find (we did not hide the report from the step groups), and over all 81 traces the oracle pointer's gain on fresh replays exceeded binary search's by only 0.06. The oracle-fix evidence was not audited, and 15 of its 60 corrections replaced the agent's write. The registered digests carried only the step; the post hoc groups added a diagnosis and a correction, but no group named a harness component to change, and all used one loop and one proposer model.

Ethics Statement

All environments use synthetic users and data. The human audit was done by two graduate students working on LLM agents, who volunteered without pay and saw only these synthetic episodes. Attribution methods that misdiagnose failures can lead self-improving agents to change the wrong behaviour; our results suggest that such diagnoses should be checked against simple baselines before they drive automatic changes.

References

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.

Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.

Joseph Y. Halpern. 2016. *Actual Causality*. MIT Press.

Yeonjun In, Md Mehrab Tanjim, Jayakumar Subramanian, Sungchul Kim, Uttaran Bhattacharya, Wonjoong Kim, Sangwu Park, Somdeb Sarkhel, and Chanyoung Park. 2026. Rethinking failure attribution in multi-agent systems: A multi-perspective benchmark and evaluation. In *International Conference on Machine Learning*. ArXiv:2603.25001.

Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-Thu Le. 2026. Towards self-improving error diagnosis in multi-agent systems. *arXiv preprint arXiv:2604.17658*.

Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. 2026. REFLECT: Intervention-supported error attribution for silent failures in LLM agent traces. *arXiv preprint arXiv:2606.09071*.

Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. 2026. Who&When Pro: Can LLMs really attribute failures in AI agents? *arXiv preprint arXiv:2607.09996*.

Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. DoVer: Intervention-driven auto debugging for LLM multi-agent systems. *arXiv preprint arXiv:2512.06749*.

Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference*, 2nd edition. Cambridge University Press.

Jaineet Shah. 2026. Causal agent replay: Counterfactual attribution for LLM-agent failures. *arXiv preprint arXiv:2606.08275*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. 2026. Accurate failure prediction in agents does not imply effective failure prevention. *arXiv preprint arXiv:2602.03338*.

Alva West, Yixuan Weng, Minjun Zhu, Zhen Lin, Zhiyuan Ning, and Yue Zhang. 2025. Abduct, act, predict: Scaffolding causal inference for automated failure attribution in multi-agent systems. *arXiv preprint arXiv:2509.10401*.

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. *arXiv preprint arXiv:2609.24972*.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026a. AgenTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*. ArXiv:2509.03312.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. ArXiv:2412.06559.

Chenyang Zhu, Spencer Hong, Jingyu Wu, Kushal Chawla, Yuhui Tang, Youbing Yin, Nathan Wolfe, Erin Babinsky, and Daben Liu. 2026a. RAFFLES: Reasoning-based attribution of faults for LLM systems. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, pages 7659–7688. ArXiv:2509.06822.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026b. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Profile Details

Replays. τ^2 replays restore the database state after the kept steps and continue with the same agent harness and user simulator (DeepSeek-V4-Flash, temperature 0); a corrected action replaces the agent's turn at step k (a message, or tool calls whose results are then computed). AppWorld replays restore the app state by re-executing the kept code and force the corrected turn, which must contain one Python block; the episode continues with the incumbent harness. A correction the replay cannot apply counts as giving no gain.

From earliest step to profile. Our first registered protocol named the earliest step whose correction succeeded on at least 2 of 4 replays more than 4 null replays did. On 20 pilot failures, independent runs of it named the same step in 42% of pairs, mostly because the oracle corrected a step in one run and not in another. Before computing any profile on the main set, we registered protocol 3 instead.

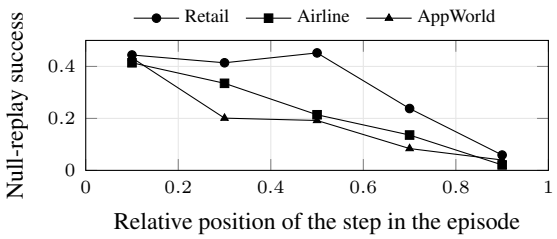


Figure 2: Success of uncorrected (null) replays from a step, by the step's position in the failed episode (fifths), at the steps where the oracle proposed a correction. Each point averages 27 to 35 steps in the first fifth and 81 to 323 in the others.

Retest. The retest failures were drawn with a seeded random choice of 10% per domain before any profile was computed and run with fresh oracle samples and replays. Of the 17 failures both runs found rescuable, the two whose best rescue step differed were one and two steps apart, and step-level gains correlated .63 to .80.

Cost. The rescue profiles cost $11.1 for the first 186 τ^2 failures and $13.7 for AppWorld (plus $0.7 for its retest); all parts of the 14 τ^2 failures added later cost about $1.5. The attribution study, including methods, search, pilots and Who&When, cost about $58, and the post hoc analyses for the main set $23.1: judges with other definitions or without grading information $9.4, the observed-action control $3.9 and the second oracle $9.8. The repair experiment (§6) cost about $35: $6.4 for drafting and critique, $1.8 for attribution and search replays, $7.9 for RRSI's evaluations and smoke tests (5,639 episodes) and $18.9 for held-out deployment (14,880 episodes); post hoc, its failures' rescue profiles cost $5.0 and the oracle-pointer group $5.9. Search with every suspect tested cost $8.2 and the full profiles without grading $3.6.

B Prompts

The τ^2 prompts follow, verbatim; braces mark what is filled in. The AppWorld prompts replace the description of the agent and the task, the grading report and the component descriptions with AppWorld's (the code agent, its unit tests, and the same nine components worded for it), and the oracle and search ask for one assistant turn with one Python block.

Shared introduction (all judges)

```
A customer-service agent (an LLM inside a harness) failed a
↪  task of tau2-bench. Below is the failure as the developers'
↪  analysis tools see it: the customer's hidden instructions,
↪  the conversation (each agent step is numbered), and the
↪  grading.

The DECISIVE MISTAKE is the earliest agent step such that, had
↪  the agent acted correctly at that step and continued
↪  normally, the task would most likely have succeeded.

Harness components (a fix to the harness changes one of these):
- prompt: the system prompt text (instructions and policy
↪  wording the model reads)
- control_flow: the agent loop code: how the next action is
↪  produced, checks or gates applied to tool calls before they
↪  run, retries, enforced confirmations
- config: numeric settings such as temperature, token limits or
↪  step limits
- output_plumbing: how tool results and tool descriptions are
↪  rendered to the model
- context_mgmt: which parts of the conversation history the
↪  model sees
- client_tool: extra tools the harness gives the model (e.g. a
↪  checker or calculator)
- skill: instruction files loaded for specific situations
- memory: structured state the harness keeps across steps
- subagent: an extra model call, e.g. a second model reviewing
↪  an action

=== FAILURE ===
{the failure: hidden instructions, numbered steps, grading
↪  report}
```

All-at-once

```
Find the decisive mistake and the harness component whose
↪  change would most likely have prevented it. Answer with
↪  JSON:
{"step": <step number>, "top3": [<the three most likely step
↪  numbers, best first>],
 "component": "<component>", "components3": ["<three most
 ↪  likely components, best first>"],
 "reason": "<one sentence>"}
```

Step-by-step (asked for $k = 0, 1, \ldots$ on the episode up to step k)

```
Judge step {k} only, given what happened before it. Is the
↪  agent's action at step {k} the decisive mistake (the
↪  earliest step whose correction would most likely have made
↪  the task succeed)? Answer with JSON:
{"decisive": true | false, "component": "<component, if
↪  decisive>", "reason": "<one sentence>"}
```

Binary search (episode up to step `hi`)

```
The decisive mistake is assumed to lie between step {lo} and
↪  step {hi}. Is it in the first half (steps {lo} to {mid}) or
↪  in the second half (steps {mid + 1} to {hi})? Answer with
↪  JSON:
{"half": "first" | "second", "component": "<the component most
↪  likely at fault>",
 "reason": "<one sentence>"}
```

Root-cause attributor

```
An LLM agent failed a task. Find the root cause of the failure.

Agent components (id: role):
{COMP_TEXT}

Trace:
{view(rec)}

Give (1) a one-sentence root cause describing the cause, not
↪  the symptom;
(2) the index of the earliest step where the agent went wrong;
(3) the single component whose change would most plausibly
↪  prevent this failure.
Answer with JSON: {"root_cause": "...", "step": <int>,
↪  "component": "<component id>"}
```

Oracle (system prompt) As run; its opening sentence predates our terminology, and the profile it builds is the oracle-dependent reference of §3, not a ground truth about where a failure began.

```
You construct ground truth for failure attribution. A
↪  customer-service agent failed a task of tau2-bench. You see
↪  the domain policy, the agent's tools, the task's grading
↪  criteria (the database must end as if exactly the gold
↪  actions were executed, and the agent must tell the user
↪  every required piece of information), and the conversation
↪  up to one step of the agent.

Decide whether the agent's action at that step is a mistake: an
↪  action a careful agent following the policy would not take
↪  on the way to the gold outcome. Actions that are correct,
↪  or harmless and do not lead away from the gold outcome, are
↪  ok.

If it is a mistake, give the single action a careful agent would
↪  take at that step instead: one assistant turn, either a
↪  message to the user or one or more tool calls. Correct only
↪  this step; do not do more than a careful agent would do at
↪  this point. The corrected action must be one the agent
↪  could have taken with what it knew then (the conversation
↪  and tool results so far, and the policy); use the grading
↪  criteria only to know what the right outcome is, never to
↪  supply ids, amounts or facts the agent had not yet learned.

Answer with JSON:
{"verdict": "ok" | "mistake", "why": "<one sentence>",
 "action": {"message": "<text to the user>"} | {"tool_calls":
 ↪  [{"name": "<tool>", "arguments": {...}}]}}
("action" only when the verdict is mistake.)
```

Counterfactual search, suspect call (system prompt)

```
A customer-service agent on tau2-bench failed its task: the
↪  customer's request was not resolved as the domain policy
↪  requires. You see the domain policy, the agent's tools and
↪  the conversation; you do not know what the correct outcome
↪  was.

Name the steps where the agent most likely went wrong, most
↪  suspect first (at most 5). For each, give the single action
↪  a careful agent would have taken at that step instead (one
↪  assistant turn: a message to the user or one or more tool
↪  calls, using only what the agent knew at that point) and
↪  the harness component most likely at fault.

Harness components:
- prompt: the system prompt text (instructions and policy
↪  wording the model reads)
- control_flow: the agent loop code: how the next action is
↪  produced, checks or gates applied to tool calls before they
↪  run, retries, enforced confirmations
- config: numeric settings such as temperature, token limits or
↪  step limits
- output_plumbing: how tool results and tool descriptions are
↪  rendered to the model
- context_mgmt: which parts of the conversation history the
↪  model sees
- client_tool: extra tools the harness gives the model (e.g. a
↪  checker or calculator)
- skill: instruction files loaded for specific situations
- memory: structured state the harness keeps across steps
- subagent: an extra model call, e.g. a second model reviewing
↪  an action

Answer with JSON:
{"suspects": [{"step": <int>, "why": "<one sentence>",
↪  "component": "<component>",
  "action": {"message": "<text>"} | {"tool_calls": [{"name":
  ↪  "<tool>", "arguments": {...}}]}}]}
```

Variants (post hoc). The reruns of the all-at-once judges and binary search in §5 use the same prompts with one sentence of the introduction replaced. Registered:

```
The DECISIVE MISTAKE is the earliest agent step such that, had
↪  the agent acted correctly at that step and continued
↪  normally, the task would most likely have succeeded.
```

Most likely rescue:

```
The DECISIVE MISTAKE is the agent step whose correction would
↪  most likely have rescued the task: had the agent acted
↪  correctly at that step and continued normally, the task
↪  would most likely have succeeded. If several steps qualify,
↪  choose the one where a correction is most likely to make
↪  the task succeed, which need not be the earliest.
```

Largest gain over null:

```
The DECISIVE MISTAKE is the agent step where a correction would
↪  make the biggest difference: the step at which acting
↪  correctly, and then continuing normally, would most
↪  increase the chance that the task succeeds compared with
↪  letting the agent act again from that step without any
↪  correction. A step from which the task would often have
↪  succeeded anyway, without a correction, is a weak choice
↪  even if a correction there would succeed.
```

The no-grading variant keeps the registered definition, removes the grading report and the customer's hidden instructions from the episode, gives the domain policy and tools (AppWorld: the agent's rules and the APIs) as counterfactual search receives them, and opens with: "… You see the domain policy and the agent's tools, but not the customer's instructions or the grading, so you do not know what the correct outcome was."

The informed suspect call of counterfactual search (post hoc) adds the customer's hidden instructions and the grading report to the episode and says "… and the grading (the gold actions and the information the agent had to give), so you know what the correct outcome was", asking that the grading be used only to know the right outcome, never to supply identifiers, amounts or facts the agent had not yet learned. The aligned call (post hoc) is the informed call with "Name the steps where the agent most likely went wrong, most suspect first" replaced by "Name the steps where a correction would make the biggest difference, best first" followed by the largest-gain definition above.

Repair experiment: notes to the proposer. RRSI's proposer prompt is unchanged. In the no-attribution group the round's analysis report is replaced by the first note below; the step groups get the second note and one digest per failing trace, {"task_id", "lens": "failure", "decisive_step": k, "step_excerpt": <the step, at most 600 characters>, "blocker": "failure decided at step k"}.

```
No analysis report this round. Read the traces yourself
↪  (list_traces, then read_trace) to find what went wrong; the
↪  failing traces are listed first.
```

τ^2	steps/dec.	utterance	read	write	transfer
Retail	1165/65	54%/28	34%/1	11%/36	1%/0
Airline	1026/37	55%/25	31%/4	12%/8	1%/0
AppWorld	steps/dec.	docs	read	write	submit
AppWorld	1961/55	27%/3	45%/9	14%/29	5%/9

Table 4: Share of all agent steps of each kind / number of best rescue steps of that kind. AppWorld "docs" looks up API documentation; 5 AppWorld best rescue steps (9% of steps) are other code.

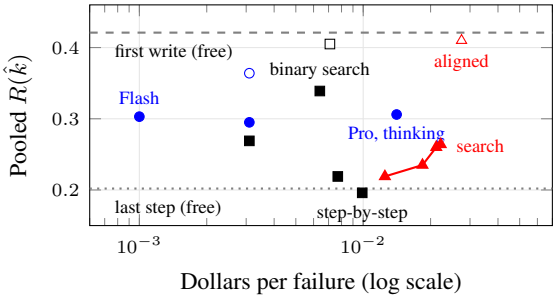


Figure 3: Rescue score against cost. Circles: all-at-once judges; squares: other LLM methods; triangles: counterfactual search at budgets 8, 16, 24 and 40. The free first-write rule lies above every registered method. Open markers (post hoc): the all-at-once judge and binary search asked for the largest gain, and aligned search.

```
No analysis report this round. Read the traces yourself
↪ (list_traces, then read_trace) to find what went wrong; the
↪ failing traces are listed first. For each failing trace,
↪ PER-TASK DIGESTS give decisive_step: the step at which an
↪ automatic failure-attribution method located the failure.
↪ Read the trace around that step (read_trace with
↪ from_step/to_step) to see what went wrong there.
```

C Secondary Metrics, Step Kinds and Search Budgets

Table 5 gives the secondary metrics, Table 4 the kinds of agent steps and best rescue steps, and Table 6 counterfactual search by replay budget. Search's suspects included a step with a gain of at least 0.5 on 122 of 157 rescuable failures (78%); its confirmations were the weak point. Of the 87 rescuable failures where its own test (4 corrected against 4 null replays) confirmed a suspect, the confirmed step had a gain of at least 0.5 in 41; elsewhere its corrections at the right step failed, or a weaker step passed the test first.

D Post-hoc Variants and Who&When

Robustness details. Choosing rescuable failures and scoring on the same noisy gains flattered the best rescue step (0.75, against 0.48 out of sample) but no method over another. The second oracle found 120 failures rescuable instead of 157 and agreed on the best rescue step for 71–72% of those both found rescuable. The screen covered the 4,795 corrections that entered a profile; it flagged an identifier, name or date the agent had not yet seen in 0.7% and two or more calls beyond the observed step in 1.8%. For the human audit we drew 20 corrections per domain with a positive gain on rescuable failures, half with a mechanical flag where enough were flagged (19 of 60 in all), spread over step kinds. Two auditors, graduate students working on LLM agents who volunteered without pay, each saw the episode before the step, the observed action, the correction and the oracle's reason, but not the flags or each other's labels, and answered three questions: does the correction use information the agent had not seen, does it do more than one careful step, and is it valid given what the agent could see and the policy. Their written instructions, released with the sample, drafted validity as two questions, whether a careful agent had reason to take the action without knowing the grading and whether the policy allowed it, and asked them to judge the first before reading the oracle's reason, which may mention the grading. The first auditor judged 4, 6 and 40 of 60 yes, the second 4, 3 and 44; 36 were valid for both and 48 for at least one. Among the 41 unflagged corrections, 28 and 30 were valid; among the 17 τ^2 corrections that change state, 6 and 10 (6 for both). Samples of one step sometimes proposed the same action, so the 60 items hold 52 distinct corrections (two appear three times and four twice): over these, 34 and 37 were valid (30 for both, 41 for either; $\kappa = .51$ for validity, .46 for scope, .30 for unseen information), and 3 and 6 of the 12 distinct τ^2 writes. During the audit both auditors adopted a lenient reading: validity judged by a correction's content, so several calls in one turn counted as valid if each could be made in turn, as the policy's one-call rule requires, and several calls of one kind as one step. The first auditor marked 11 labels changed under this reading (from invalid to valid, or from more than one step to one), the second 12 labels given under it; the release marks both. Most were airline items (8 of 11 and 7 of 12). Counting every label given under this reading as invalid, 29 and 34 of 60 corrections were valid (26 for both; $\kappa = .64$), 4 and 6 of the 17 τ^2 corrections that change state, and 27 and 29 of the 52 distinct corrections. The labels do not enter the re-scoring, which uses mechanical filters, so the reading changes these shares, not the rankings.

Method	Model	Exact [95%]	Within 1	Earliest	Share of max	$R(\hat{k})$, all failures
First write	rule	0.38 [0.27, 0.49]	0.62	0.33	0.52	0.23 [0.17, 0.30]
Last step	rule	0.13 [0.07, 0.20]	0.43	0.08	0.25	0.11 [0.07, 0.16]
All-at-once	Flash	0.28 [0.20, 0.36]	0.52	0.31	0.40	0.19 [0.14, 0.23]
All-at-once	Pro	0.31 [0.22, 0.39]	0.52	0.32	0.39	0.17 [0.13, 0.22]
All-at-once	Pro, thinking	0.32 [0.22, 0.42]	0.60	0.36	0.42	0.18 [0.14, 0.23]
Step-by-step	Pro	0.19 [0.13, 0.26]	0.38	0.22	0.25	0.11 [0.07, 0.15]
Binary search	Pro	0.32 [0.24, 0.40]	0.58	0.34	0.43	0.19 [0.15, 0.24]
Root-cause attributor	Pro	0.28 [0.19, 0.37]	0.49	0.31	0.36	0.16 [0.12, 0.20]
RRSI trace digester	Pro	0.21 [0.14, 0.29]	0.41	0.22	0.29	0.14 [0.10, 0.17]
Counterfactual search@40	Pro, replays	0.26 [0.18, 0.35]	0.44	0.31	0.35	0.15 [0.11, 0.19]

Table 5: Secondary metrics, pooled over the three domains. Exact, within one step, earliest rescuing step and share of the largest gain are over rescuable failures; the last column is over all 300 failures.

Budget	Retail	Airline	AppWorld	Pooled	$
8	0.22	0.13	0.28	0.22	0.0125
16	0.21	0.23	0.27	0.23	0.0184
24	0.22	0.31	0.27	0.26	0.0212
32	0.22	0.31	0.28	0.26	0.0222
40	0.22	0.31	0.28	0.26	0.0222

Table 6: Counterfactual search: mean $R(\hat{k})$ on rescuable failures by replay budget, read off the same runs, and dollars per failure.

Both accepted a search that used common knowledge (a New York airport code) the agent had not seen. The second also accepted small parameter errors (an extra argument, a page size above the API's limit), changes to orders whose status no longer allowed them, and twice the user's earlier request over a later confirmation; the first rejected these. Invalid τ^2 corrections most often replaced the product, payment, reason or baggage count the user had last confirmed, or modified an order whose status no longer allowed it; invalid AppWorld corrections called APIs that do not exist, passed parameters the APIs reject or computed the wrong answer. Twelve items (9 distinct corrections) lie at the steps the first write names: of its 7 distinct τ^2 corrections the auditors judged 1 and 3 valid, of its 2 AppWorld ones both. The filters were written after reading the auditors' reasons, so their agreement with them is in sample. On the 12 distinct audited τ^2 writes, the direct filter, which drops a correction that changes the agent's write at that step and loses a value (an identifier, option or reason, as text) that the user wrote or the agent showed in the exchange the user answered last (it misses values shown only by description and numbers such as a bag count), agreed with the auditors on 7 and 8 and dropped none the first judged valid; dropping every replacement of the agent's write agreed on 9 and 8, and also dropping several calls in one turn on 10 and 9, although both auditors accepted such calls when each could be made in turn. The direct filter dropped 236 of 3,103 τ^2 corrections (144 retail, 92 airline) and left 44 retail, 36 airline and 55 AppWorld failures rescuable; the first write scored 0.24 in retail, 0.23 in airline and 0.30 pooled, fifth, with binary search at 0.32 and the all-at-once judge with thinking at 0.34 (-0.04 $[-0.13, +0.06]$; on the fixed cohort fifth, -0.04 $[-0.12, +0.05]$). Dropping every replacement dropped 330 and left 39, 32 and 55 rescuable; the first write scored 0.14 in retail, 0.18 in airline and 0.26 pooled, with binary search at 0.28 and counterfactual search at 0.29 (on the fixed cohort seventh, -0.08 $[-0.16, +0.01]$). Also dropping several calls in one turn (400) gave the same scores to within 0.01. Dropping every τ^2 correction that changes state (950), an upper bound that also drops valid bookings, left 24, 26 and 55 rescuable and the first write at 0.27, -0.08 $[-0.18, +0.04]$ below the all-at-once judge with thinking (0.34). The auditors' invalid AppWorld corrections were API errors, not choices the grading could guide, so we keep AppWorld's writes in these filters; dropping them as well (433 of 1,692, submissions included) left 81 failures rescuable and the first write eighth (0.17, -0.16 $[-0.27, -0.04]$ below the judge; on the fixed cohort -0.11 $[-0.18, -0.05]$) (`attrib/audit_rescore.py`). In the repair experiment, the 60 oracle fixes given without reading held 25 messages and 35 tool calls; 21 changed state, 15 of them replacing the agent's write at that step (6 changing a choice the user had just confirmed), and 8 made several calls in one turn (`attrib/oraclefix_screen.py`). With the observed action forced as the control, scored on the

Reference	Resc.	First wr.	Best LLM	Rank	τ
Registered	157	0.42	Binary search 0.34	1	1.00
$K = 2$	188	0.37	Binary search 0.30	1	0.89
$K = 1$	181	0.39	Binary search 0.33	1	0.82
$\theta = 0.25$	226	0.31	Binary search 0.26	1	0.91
$\theta = 0.375$	177	0.38	Binary search 0.31	1	0.96
$\theta = 0.625$	116	0.50	Binary search 0.38	1	0.78
$\theta = 0.75$	107	0.54	Binary search 0.39	1	0.82
One rescuing step	102	0.33	Binary search 0.27	1	1.00
No null replays	203	0.37	Judge, thinking 0.42	6	0.60
No null replays	157†	0.46	Judge, thinking 0.44	1	0.87
Split samples	193	0.34	Binary search 0.27	1	0.96
Observed action as control	157†	0.44	Judge, thinking 0.40	1	0.82
Second oracle (Flash)	120	0.38	Binary search 0.29	1	0.78
Both oracles averaged	121	0.43	Binary search 0.32	1	0.82
Flagged corrections dropped	152	0.43	Binary search 0.34	1	1.00
Oracle without grading	157†	0.05	Counterfactual search@40 0.11	7	0.24
Confirmed τ^2 choices changed, dropped	135	0.30	Judge, thinking 0.34	5	0.69
Confirmed τ^2 choices changed, dropped	157†	0.26	Judge, thinking 0.30	5	0.64
Replaced τ^2 writes dropped	126	0.26	Judge, thinking 0.34	8	0.38
Replaced τ^2 writes dropped	157†	0.21	Judge, thinking 0.29	7	0.47
Replaced or multi-call τ^2 writes dropped	125	0.26	Judge, thinking 0.34	7	0.38
All τ^2 writes dropped	105	0.27	Judge, thinking 0.34	7	0.38
All writes dropped	81	0.17	Judge, thinking 0.33	8	0.20
All writes dropped	157†	0.10	Judge, thinking 0.21	8	0.29

Table 7: Method scores under other reference choices (post hoc), pooled over the three domains: rescuable failures (re-selected under each choice; †: the registered 157 kept as a fixed cohort), the first write's mean $R(\hat{k})$, the best LLM method and its $R(\hat{k})$ (judge, thinking: all-at-once with Pro and thinking), the first write's rank among the ten methods of Table 1, and Kendall's τ of the method ordering against the registered one ($K = 4$ oracle samples, null replays, threshold $\theta = 0.5$). Rows with fewer samples average over all subsets of the four; split samples choose rescuable failures on two samples and two null replays and score on the others; the observed action forced at step k replaces null replays and the oracle without grading replaces the registered oracle, both scored on the fixed cohort; the second oracle is V4-Flash, "both oracles averaged" averages the two oracles' profiles, and "flagged corrections dropped" removes the corrections the admissibility screen flagged for an unseen value or two more calls. The last rows drop, after the human audit, τ^2 corrections that change a choice the user had just stated or confirmed (the direct filter), those that replace the write the agent made at that step, those and corrections that make several calls in one turn, every τ^2 correction that changes state, and every correction that changes state in any domain (AppWorld's submissions included); rows marked † repeat a choice on the fixed cohort.

fixed cohort of 157 registered rescuable failures (156 of which would also be rescuable under this control), the all-at-once judge with Pro rose from 0.30 to 0.39, leaving the first write ahead of the best registered judge by 0.04 $[-0.07, +0.15]$. The second oracle proposed corrections less often (0.16–0.26 of samples against 0.22–0.37), and the two oracles agreed on rescuability for 69–80% of failures ($\kappa = .36$–$.61$). Under its profile the first write led binary search by $+0.08$ $[+0.01, +0.16]$, and under the average of both profiles by $+0.10$ $[+0.02, +0.19]$ ($\tau = .82$); binary search asked for the largest gain came within 0.02 of it under both. The admissibility screen also flagged a number not seen before in 6.1% of corrections, mostly sums or differences the agent could compute; corrections of retail writes, the most effective kind, were flagged for an unseen value in none and for any unseen value in 0.4%. Without the corrections flagged for an unseen value or two more calls, 152 failures were rescuable, the first write scored 0.43 and binary search 0.34 ($+0.09$ $[-0.01, +0.19]$), and binary search asked for the largest gain came within 0.02 ($+0.02$ $[-0.07, +0.10]$). Without null replays a step scores $R_k + (m_k/K)\,\hat{p}_k^{\text{null}}$, where m_k of the K samples corrected it; at the registered rescuable failures the added term averaged 0.08 to 0.13 for the LLM methods and 0.04 for the first write.

Table 8 compares the judges' registered prompts with the re-prompted variants (Appendix B), Table 9 a rule naming the first deviation from τ^2's gold actions, which the judges read (0.50, though the oracle corrects from the same gold actions), Tables 10 and 11 split rescue gains into how often the oracle proposed a correction and how well its corrections worked, and Table 12 screens the corrections for information the agent had not seen and for scope.

Who&When. To check that our judges are not unusually weak, we ran the all-at-once, step-by-step and binary-search judges on the 184 logs of Who&When with its authors' prompts and the setting that shows the judge the task's correct final answer (Who&When's "ground truth", not an attribution label) (Zhang et al., 2025). They identified the responsible agent in 58–67% of logs with all-at-once judging and the responsible step in 3–25% (other judges 4–32%; Table 13), the same pattern the benchmark reports: agents are found far more often than steps.

E Repair Candidates

The proposer saw 81 failing traces, 42 of them retail. Held-out deployment used 240 episodes per harness (retail 40 tasks $\times$ 6, airline 20 $\times$ 12). The equivalence margin of ± 2.5 points is about one standard deviation of a single candidate's gain in earlier runs of the loop. Figure 4 shows the held-out gain of every deployable candidate in the repair experiment.

F Further Checks

All analyses and runs in this section are post hoc.

Method	Definition	Retail	Airline	AppWorld	Pooled [95%]	Exact	Position	$	Difference [95%]
All-at-once, Flash	earliest (reg.)	0.29	0.41	0.25	0.30 [0.24, 0.37]	0.28	0.64	0.0010	
	most likely rescue	0.36	0.35	0.26	0.32 [0.25, 0.39]	0.32	0.68	0.0010	+0.02 [−0.04, +0.08]
	largest gain over null	0.29	0.41	0.28	0.31 [0.24, 0.39]	0.34	0.64	0.0010	+0.01 [−0.04, +0.06]
All-at-once, Pro	earliest (reg.)	0.30	0.34	0.26	0.30 [0.23, 0.36]	0.31	0.63	0.0031	
	most likely rescue	0.33	0.47	0.33	0.36 [0.29, 0.44]	0.38	0.65	0.0031	+0.07 [+0.01, +0.12]
	largest gain over null	0.39	0.42	0.29	0.36 [0.29, 0.44]	0.36	0.67	0.0031	+0.07 [+0.02, +0.12]
	earliest, no grading	0.17	0.32	0.25	0.23 [0.17, 0.29]	0.24	0.58	0.0028	−0.07 [−0.13, −0.00]
Binary search, Pro	earliest (reg.)	0.33	0.34	0.35	0.34 [0.27, 0.41]	0.32	0.69	0.0064	
	most likely rescue	0.38	0.42	0.32	0.37 [0.29, 0.44]	0.35	0.72	0.0069	+0.03 [−0.04, +0.10]
	largest gain over null	0.44	0.42	0.35	0.40 [0.33, 0.48]	0.39	0.74	0.0071	+0.07 [−0.01, +0.13]
	earliest, no grading	0.21	0.24	0.24	0.23 [0.16, 0.29]	0.20	0.66	0.0064	−0.11 [−0.20, −0.02]
Counterfactual search@0	first suspect, no grading	0.22	0.13	0.28	0.22 [0.16, 0.28]	0.20	0.52	0.0037	
Counterfactual search@40	no grading	0.22	0.31	0.28	0.26 [0.20, 0.33]	0.26	0.57	0.0222	+0.04 [+0.00, +0.09]
First write	rule	0.53	0.27	0.39	0.42 [0.33, 0.51]	0.38	0.78	0	

Table 8: Judges with their registered prompt (earliest decisive mistake) and post hoc variants: the step most likely to rescue the task, the step with the largest gain over letting the agent act again, and the registered definition without grading information or hidden instructions (what counterfactual search sees); and search with and without its replays. On the 157 rescuable failures: mean $R(\hat{k})$, exact agreement with the best rescue step, mean relative position of the named step, dollars per failure (mean over domains), and the paired difference in pooled $R(\hat{k})$ from the registered prompt (search@40: from search@0) with a cluster-bootstrap interval.

Method	Retail (65)	Airline (37)	τ^2 (102) [95%]	Exact	Gold deviation − method [95%]
Gold deviation (rule)	0.58	0.35	0.50 [0.38, 0.60]	0.49	
First write (rule)	0.53	0.27	0.44 [0.32, 0.55]	0.43	+0.06 [+0.01, +0.12]
All-at-once, Pro	0.30	0.34	0.32 [0.23, 0.40]	0.33	+0.18 [+0.02, +0.33]
Binary search, Pro	0.33	0.34	0.33 [0.24, 0.43]	0.34	+0.16 [+0.02, +0.30]
Counterfactual search@40	0.22	0.31	0.25 [0.17, 0.34]	0.25	+0.24 [+0.09, +0.38]

Table 9: The gold-deviation rule (post hoc): the first state-changing call that matches no unused gold action, else the last step (20% of failures). Mean $R(\hat{k})$ on the 102 rescuable τ^2 failures and paired differences with cluster-bootstrap intervals.

Method	Model	Proposed	Gain if proposed	$R(\hat{k})$
First write	rule	0.64	0.61	0.42
Last step	rule	0.53	0.33	0.20
All-at-once	Flash	0.58	0.51	0.30
All-at-once	Pro	0.60	0.48	0.30
All-at-once	Pro, thinking	0.61	0.49	0.31
Step-by-step	Pro	0.52	0.34	0.20
Binary search	Pro	0.64	0.49	0.34
Root-cause attributor	Pro	0.56	0.46	0.27
RRSI trace digester	Pro	0.57	0.34	0.22
Counterfactual search@40	Pro, replays	0.59	0.44	0.26
Best rescue step		0.88	0.87	0.76

Table 10: $R(\hat{k})$ split into the share of oracle samples that proposed an applicable correction at the named step and the mean gain of those corrections (over failures with at least one), on the 157 rescuable failures (post hoc).

Domain	Step	Steps	Proposed	Gain if proposed	Null
Retail	utterance	630	0.35	0.13	0.25
	read	400	0.24	0.02	0.45
	write	125	0.51	0.52	0.04
Airline	utterance	569	0.38	0.13	0.16
	read	321	0.33	0.08	0.31
	write	127	0.48	0.18	0.02
AppWorld	docs	535	0.07	0.20	0.13
	read	885	0.13	0.12	0.13
	write	272	0.53	0.21	0.13
	submit	91	0.65	0.32	0.01

Table 11: The same split over all steps of all 300 failures by kind of step, with the success of null replays where they were run (post hoc; τ^2 transfers and other AppWorld code omitted).

Admissibility by kind of step. At the first write's steps on rescuable failures, no correction used an identifier, name or date the agent had not seen, 0 to 4.5% used an unseen number (airline writes the most), and 0 to 3% added two or more calls. At best rescue steps one of 553 corrections used an unseen identifier (an AppWorld write); two or more extra calls were most common at airline utterances (12% of 85), where a correction replaced a message by several lookups, and absent at retail steps.

Intervention scope in AppWorld. A correction replaces one code block by another of similar size: 5.4 lines against 5.3 in the observed block (median 3, 90th percentile 13) and 1.04 API calls against 0.96; 97% have at most one call more than the ob-

Domain	Corrections scored	Unseen value	Unseen number	More calls ≥+2	More calls ≥+1	Calls obs. / corr.
Retail	1,554	0.4	7.6	0.4	1.5	0.94 / 0.68
Airline	1,549	1.6	10.0	2.3	3.7	1.30 / 0.80
AppWorld	1,692	0.2	1.1	2.7	15.9	0.96 / 1.04
At best rescue steps	553	0.2	2.9	2.9	5.8	

Table 12: Mechanical screen of the oracle's corrections (post hoc): every correction that entered a registered profile, in %. Unseen value: an argument (an identifier, name, e-mail or date; in AppWorld a value passed to an API to look something up) that appears neither in the episode before the step nor in the policy, tool schemas or API documentation. Unseen number: a number above 10 likewise absent, often a sum or difference the agent could compute. More calls: tool or API calls beyond $\max(1, \text{observed})$. Re-scored without the corrections flagged for an unseen value or two more calls, the ranking was unchanged (Table 7).

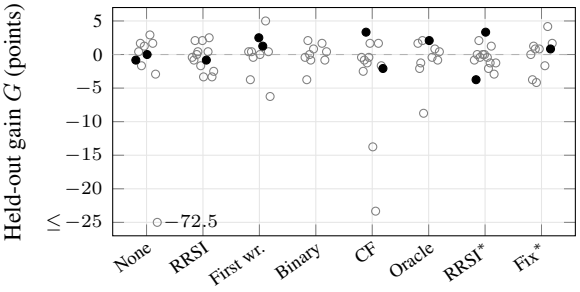


Figure 4: Held-out gain of every deployable candidate in the repair experiment, by the evidence its proposer got (*: no reading of failing traces; Fix: oracle step, diagnosis and correction). Filled: accepted by RRSI's own rule. One candidate (-72.5) is drawn at -25.

served block (or than one call). At best rescue steps 78% of a correction's lines are new. A corrected block thus often does more than a conversational turn, but no more than the agent's own blocks do.

Individual profiles. Choosing the best rescue step on two oracle samples and two null replays and re-choosing it on the other two (all 12 splits, all 300 failures), the halves agreed for 56% of the failures both found rescuable (72% within one step; 52–58% by domain), and the step chosen on one half gained 0.48 on the other against 0.75 in sample. With all four samples, two independent runs agreed for 15 of 17 failures. Profiles therefore rank methods consistently while individual best steps carry noticeable selection noise.

Position and sampling noise (post hoc). Mean rescue gain rose with a step's relative position, from 0.00 in the first tenth of an episode to 0.26 at 80–90% and 0.21 in the last tenth. Scored against the mean gain of steps in the same tenth, the second-to-last step scored 0.10, the first write 0.24 and binary search, the best registered judge, 0.19 (first write $+0.05$ $[-0.04, +0.14]$); binary search asked for the gain scored 0.24 and aligned search 0.26, and the registered methods kept their order ($\tau = .87$). Dividing each gain by the room uncorrected replays leave, $R_k/(1-\hat{p}_k^{\text{null}})$, also kept the first write first (0.43 against 0.35; $\tau = .96$). In task-clustered bootstrap samples of the 157 failures the first write ranked first among the ten registered methods in 94% of samples, in 63% with the two Pro judges asked for the gain (binary search 33%) and in 50% with aligned search added (aligned search 29%). Kendall's τ between a sample's ordering of the ten registered methods and the registered ordering had median 0.82 and 5th percentile 0.60, so the τ values of Table 7 lie within sampling noise; only the first place is stable (`attrib/baselines.py`).

Independent selection and scoring, and oracle variability (post hoc). Splitting each step's four oracle samples and four null replays into halves (6×2 splits), we counted failures as rescuable on one half (193 on average, more than with all four samples because a two-sample gain is coarser) and scored every method on the other. The first write ranked first among the 15 methods that cover all failures (0.34), ahead of binary search asked for the gain (0.33; $+0.01$ $[-0.06, +0.08]$), the all-at-once judge asked for the gain (0.29), the second-to-last step (0.27) and registered binary search (0.27; $+0.07$ $[-0.01, +0.15]$); on the 150 such failures among the registered 157, aligned search scored 0.41 against the rule's 0.42 ($+0.01$ $[-0.09, +0.10]$). Scored on the held-out half over all 300 failures, the rule and gain-asked binary search scored 0.23 each. The intervals in the body resample tasks and hold each failure's profile fixed. A two-level bootstrap (2,000 draws) that also redraws, at every step, the four oracle samples with their corrected replays and the four null replays, then recomputes profiles, the rescuable set and all scores, widened the first write's lead over binary search from $[-0.01, +0.18]$ (tasks only, same draws) to $[-0.03, +0.18]$, and its lead over gain-asked binary search from $[-0.07, +0.10]$ to $[-0.09, +0.10]$; the rule ranked first among registered methods in 86% of draws instead of 94%. Re-selection on redrawn profiles admitted more failures (172 on average), since a maximum over noisy gains is biased upward; holding the regis-

Method	Model	Hand-crafted (58) agent	Hand-crafted (58) step	Generated (126) agent	Generated (126) step
All-at-once	Flash	0.59	0.09	0.58	0.21
All-at-once	Pro	0.62	0.03	0.67	0.18
All-at-once	Pro, thinking	0.62	0.15	0.65	0.25
Step-by-step	Pro	0.24	0.19	0.44	0.32
Binary search	Pro	0.26	0.09	0.38	0.04

Table 13: Who&When, with the task's correct answer shown to the judge as in its authors' setting: accuracy of the responsible agent and step.

Evidence	Accepted	Gains of accepted	Round gain	− RRSI	95% interval states	95% interval runs	p runs
No attribution	2/8	$-0.8, +0.0$	-0.10	$+0.00$	$[-0.37, +0.37]$	$[-0.54, +0.54]$	1.00
RRSI analysis	1/8	-0.8	-0.10				
First write	2/8	$+2.5, +1.2$	$+0.47$	$+0.57$	$[-0.32, +1.46]$	$[-0.49, +1.63]$	0.50
Binary search	0/8	–	$+0.00$	$+0.10$	$[-0.14, +0.35]$	$[-0.23, +0.44]$	1.00
Counterfactual search@40	2/8	$+3.3, -2.1$	$+0.16$	$+0.26$	$[-0.99, +1.51]$	$[-1.52, +2.04]$	0.75
Oracle step	1/8	$+2.1$	$+0.26$	$+0.36$	$[-0.27, +0.99]$	$[-0.80, +1.52]$	1.00
RRSI analysis, no reading	2/8	$-3.8, +3.3$	-0.05	$+0.05$	$[-1.55, +1.66]$	$[-2.52, +2.63]$	1.00
Oracle fix, no reading	1/8	$+0.8$	$+0.10$	$+0.21$	$[-0.11, +0.53]$	$[-0.17, +0.59]$	0.50
Any step pointer − RRSI				$+0.33$	$[-0.13, +0.78]$	$[-0.39, +1.04]$	0.25
Oracle evidence − RRSI, pooled				$+0.26$	$[-0.35, +0.87]$	$[-0.49, +1.01]$	0.50
Oracle fix − RRSI, no reading				$+0.16$	$[-1.44, +1.76]$	$[-2.16, +2.47]$	0.75

Table 14: The accepted round in the repair experiment: rounds of 8 in which RRSI's own rule accepted a candidate, those candidates' held-out gains, the round's gain (the accepted candidate's, 0 if none) averaged over states, and its difference from RRSI's analysis with 95% t intervals over the 8 states and over the 4 runs (each run's two checkpoints averaged first), and the exact sign-flip p over runs. Lower block: pooled groups. Points.

Best rescue step	n	First write	Registered AAO	Registered BS	Registered CF	Post hoc AAO_g	Post hoc BS_g	Post hoc CF_i
Retail, utterance	28	0.24	0.36	0.31	0.25	0.47	0.41	0.39
Retail, write	36	0.76	0.25	0.35	0.19	0.34	0.47	0.30
Airline, utterance	25	0.23	0.35	0.36	0.40	0.40	0.38	0.43
Airline, write	8	0.53	0.39	0.39	0.12	0.58	0.73	0.31
AppWorld, other code	5	0.28	0.65	0.53	0.38	0.40	0.50	0.65
AppWorld, read	9	0.28	0.10	0.18	0.42	0.11	0.18	0.35
AppWorld, submission	9	0.56	0.14	0.28	0.30	0.27	0.33	0.21
AppWorld, write	29	0.42	0.31	0.42	0.24	0.37	0.41	0.36
Best step changes state	82	0.60	0.27	0.37	0.21	0.37	0.46	0.31
Other best steps	75	0.23	0.32	0.30	0.32	0.36	0.34	0.39
Cells weighted equally	8	0.41	0.32	0.35	0.29	0.37	0.43	0.37

Table 15: Mean $R(\hat{k})$ on the 157 rescuable failures by the kind of their best rescue step (cells with at least 5 failures). AAO, BS: all-at-once and binary search with Pro; CF: counterfactual search@40; $_g$: asked for the largest gain over null; $_i$: given the grading information. State-changing steps are writes and AppWorld's submission; the last row weights the eight cells above equally.

Method	Model	All three Grading	All three None	No grading, by domain Retail	Airline	AppW.
First write	rule	0.42	0.05	0.01	0.07	0.10
Last step	rule	0.20	0.04	0.02	0.01	0.07
All-at-once	Flash	0.30	0.07	0.03	0.20	0.04
All-at-once	Pro	0.30	0.08	0.04	0.14	0.08
All-at-once	Pro, thinking	0.31	0.08	0.03	0.21	0.06
Step-by-step	Pro	0.20	0.03	0.01	0.06	0.04
Binary search	Pro	0.34	0.06	0.02	0.10	0.09
Root-cause attributor	Pro	0.27	0.05	0.02	0.19	0.01
RRSI trace digester	Pro	0.22	0.07	0.02	0.16	0.05
Counterfactual search@40	Pro, replays	0.26	0.11	0.06	0.23	0.09
All-at-once, gain	Pro	0.36	0.10	0.07	0.22	0.07
Binary search, gain	Pro	0.40	0.09	0.06	0.15	0.09
All-at-once, no grading	Pro	0.23	0.07	0.02	0.15	0.08
Binary search, no grading	Pro	0.23	0.04	0.03	0.10	0.03
CF search@0	Pro	0.22	0.08	0.07	0.08	0.08
CF search@0, grading	Pro	0.27	0.05	0.03	0.04	0.08
CF search@40, grading	Pro, replays	0.35	0.11	0.07	0.20	0.09

Table 16: Mean $R(\hat{k})$ on the 157 rescuable failures under the registered oracle (grading) and under the same oracle without the grading (none), scored at the same named steps (post hoc). Lower block: the post hoc variants of Table 1.

tered 157 fixed, the rule's lead over aligned search was $[-0.10, +0.11]$ (`attrib/independent.py`).

Oracle without grading. The oracle got the registered instructions with the grading clauses removed, the same context (policy and tools, or the agent's rules and APIs) and the episode up to the step, and was told that the episode failed; it ran at every step any method of Table 1 named on the rescuable failures and at their best and earliest rescuing steps, with 4 samples, 2 corrected replays each and the registered null replays (new ones where the registered oracle had never corrected the step). At

	All 81 traces		33 rescuable	
Pointer	Profile	Fresh	Profile	Fresh
Oracle step	0.39	0.27	0.74	0.59
First write	0.12	0.15	0.27	0.31
Binary search	0.19	0.21	0.41	0.42
CF search@40	0.13	0.12	0.29	0.25
RRSI (earliest cited)	0.10	0.09	0.26	0.16

Table 17: Rescue gain of each repair group's pointers in the registered profile of the experiment's failing traces and in fresh replays: 4 new oracle samples with 2 corrected replays each and 4 new null replays per step (8 at oracle steps); post hoc.

best rescue steps it proposed a correction in 42% of samples, against 88% with the grading. Only 34 of the 157 failures had a scanned step with a gain of at least 0.5, and its best scanned step was the registered best rescue step for 29%. The first write scored below binary search asked for the largest gain (-0.04 $[-0.08, -0.003]$, unadjusted), the all-at-once judge so asked (-0.05 $[-0.10, -0.01]$) and informed search (-0.05 $[-0.10, -0.01]$), and did not differ detectably from registered binary search (-0.01 $[-0.04, +0.02]$). It scored near zero in retail (0.01) but still highest in AppWorld (0.10, against 0.09 for search and binary search), whose writes kept a quarter of their gain.

Fresh replays in the repair experiment. The oracle pointers lost 0.12 $[0.06, 0.17]$ of their gain on fresh replays across all 81 traces (0.39 to 0.27) but stayed above RRSI's evidence ($+0.18$ $[+0.11, +0.27]$) and binary search's ($+0.06$ $[-0.01, +0.14]$; on rescuable failures $+0.17$ $[+0.01, +0.34]$); pointers chosen without the profile moved by -0.02 to $+0.02$ on all traces. The correction given to the oracle-fix group, replayed 8 more times at its step against 8 new null replays, succeeded in 0.80 of replays against 0.95 in the profile, with null success 0.15 against 0.10: its gain fell from 0.85 to 0.65 (-0.19 $[-0.27, -0.11]$), stayed at least 0.5 on 70% of the 60 traces and above 0 on 83%.

Drafts' targets. Each RRSI draft lists, for each edit, the training tasks it expects to fix. For the 88 drafts that reached RRSI's full evaluation on training tasks (both seeds), we compared each task's success under the draft with the incumbent's on the same tasks and seeds. Drafts named 2.6 to 3.8 failing tasks each; given the oracle's evidence they named 35–42% of the traces that carried it, against 22–28% from RRSI's analysis (pooled groups, over states: $+16$ points $[-6, +39]$). Per draft, failing tasks it named improved 3.3 points less than those it did not ($[-10.0, +3.5]$, t over the 87 drafts with both), 12.5 less for the 20 drafts given oracle evidence ($[-26.0, +1.0]$) and $+0.5$ $[-12.2, +13.2]$ for the 27 from RRSI's analysis. Every group lost 6 to 15 points on the incumbent's passing training tasks; a draft's gain on failing tasks correlated with its held-out gain at Spearman .19 ($p = .08$), its loss on passing tasks not at all ($-.08$).

Slots without a draft (post hoc). Twelve of the 80 registered slots produced no draft (3 without attribution, 4 first write, 2 binary search, 3 counterfactual search) and count as no gain. Over the slots with a draft, the contrasts with RRSI's analysis were $+0.55$ $[-0.53, +1.62]$ points (first write), $+0.29$ $[-0.82, +1.39]$ (binary search) and -4.35 $[-11.64, +2.95]$ (counterfactual search), and those with no attribution $+4.56$, $+4.30$ and -0.34, all with intervals spanning zero (`attrib/phaseb_proposals.py`).

Search with every suspect tested. On each rescuable failure every suspect with an applicable correction got 4 corrected and 4 null replays, reusing the registered and informed runs' replays and completing the suspects they had not tested (at most 40 replays); the answer is the suspect with the largest estimated gain, corrected minus null successes over four, the earlier suspect on ties. The registered stopping rule applied to the same replays reproduces the registered scores (0.26 and 0.35; Table 19). The aligned suspect call is given in Appendix B; aligned search scored 0.44 in retail (first write 0.53), 0.48 in airline (best judge 0.42) and 0.32 in AppWorld (first write 0.39), with 28 replays per failure on average. The three variants cost $8.2: $4.3 for the aligned one and $3.9 to complete the other two.

Full profiles without grading. On a seeded random 100 of the 300 failures (34 retail, 33 airline, 33 AppWorld, drawn without regard to rescuability), the oracle without grading ran at every step with the settings above, reusing the steps already scanned. It found 17 failures rescuable (6, 6 and 5), the registered oracle 51 (21, 12 and 18); 13 were rescuable under both, and for 54% of these the two oracles chose the same best rescue step. Its best rescue steps were utterances (8) and reads (4) in τ^2 and writes in AppWorld (5); the registered oracle's were writes for 26 of its 51 and submissions

		Registered oracle		Without grading	
Method	Model	Rescuable	All	Rescuable	All
First write	rule	0.38	0.21	0.17	0.04
Last step	rule	0.20	0.10	0.14	0.02
All-at-once	Flash	0.20	0.12	0.24	0.04
All-at-once	Pro	0.25	0.13	0.28	0.05
All-at-once	Pro, thinking	0.28	0.14	0.31	0.04
Step-by-step	Pro	0.12	0.06	0.10	0.01
Binary search	Pro	0.33	0.17	0.28	0.06
Root-cause attributor	Pro	0.19	0.10	0.24	0.04
RRSI trace digester	Pro	0.16	0.09	0.24	0.03
Counterfactual search@40	Pro, replays	0.22	0.11	0.36	0.06
All-at-once, gain	Pro	0.32	0.16	0.32	0.07
Binary search, gain	Pro	0.37	0.21	0.26	0.06
All-at-once, no grading	Pro	0.26	0.14	0.37	0.06
Binary search, no grading	Pro	0.21	0.11	0.25	0.05
CF search@0	Pro	0.18	0.10	0.32	0.05
CF search@0, grading	Pro	0.20	0.10	0.22	0.05
CF search@40, grading	Pro, replays	0.25	0.13	0.29	0.07

Table 18: Full profiles on a random 100 of the 300 failures (34 retail, 33 airline, 33 AppWorld; post hoc): mean $R(\hat{k})$ under the registered oracle and under the oracle without grading, on the failures each finds rescuable (51 and 17) and on all 100. Lower block: post hoc variants of Table 1.

Suspect call	First flip	Max. gain	Replays
Registered call	0.26	0.29	33.9
With grading info.	0.35	0.38	27.2
With grading info., gain	0.37	0.41	27.8

Table 19: Counterfactual search with every suspect tested (4 corrected and 4 null replays each, at most 40) on the 157 rescuable failures (post hoc): mean $R(\hat{k})$ when it answers the first suspect that beats null by two successes (the registered rule, first suspect if none) or the suspect with the largest estimated gain (max. gain), and replays per failure. Suspect calls: registered (no grading information), with the judges' grading information, and with it asked for the steps of largest gain over letting the agent act again. For comparison, the first write scored 0.42 and binary search asked for the largest gain 0.40 (Table 1).

for 2. On the 17, the first write scored 0.17 and ranked eighth of the ten registered methods, below counterfactual search (0.36; difference -0.19 $[-0.38, +0.00]$) and binary search (0.28; -0.11 $[-0.27, +0.04]$); on all 100 every method scored 0.01 to 0.06 and the first write ranked sixth. The ordering of the ten methods kept little of the registered one on the same failures ($\tau = .27$ on each oracle's rescuable failures, .47 on all 100). With 17 rescuable failures these rankings are descriptive. The runs cost $3.6.

Supplementary material. The supplement holds both registrations with their addenda and a timeline of commit dates, every replay's outcome (failure, step, sample, kind of replay, success, whether the correction was applied), the profiles, every method's named steps, the repair experiment's rounds and deployments, and the code that computes every table from them.

G Registration and Deviations

The registration (methods, prompts, metrics, the primary comparison and the budget) was committed before any method ran on the main set; the registrations are commits in the project repository, whose dates the committer sets, not entries in a third-party registry. Changes to the attribution study, each logged before the data it affected were analysed:

- **Reference protocol.** The registration left the protocol to be chosen on the pilot by test-retest agreement. After the earliest-step protocol agreed on 42% of pilot pairs, protocol 3 and the rescue-gain metrics replaced it (Addendum 1, committed 43 minutes after the methods started on the main set, whose outputs do not depend on the reference), before any main-set profile existed.
- **Retail top-up.** The first loop's harnesses yielded 86 distinct retail failures under the sampling rule; 14 more were drawn with the same rule from two further runs of the loop, as registered. All methods moved by at most 0.02 when they were added.
- **AppWorld.** Registered in Addendum 2 before any AppWorld failure was drawn. Two corrected replays of one step failed on every attempt because the oracle's code passed a literal ellipsis that AppWorld's request log cannot serialise; they count as corrections not applied. That failure's other corrections at the step rescued nothing and its largest gain was 0.375, so they could not have made it rescuable.

Repair experiment. Registered separately, before any attribution was run on its traces and before any candidate was drafted. As checks, the RRSI-analysis group's mean gain (-0.34) matched that of the candidates the original rounds had drafted from the same evidence (-0.42), and re-deploying each incumbent moved its held-out success by -2.5 to $+3.3$ points against its earlier deployment, within the noise of 240 episodes. The first state served as a pilot; it exposed no bug, so its cells

were kept, as registered. Two operational problems had no effect on data: a lookup by abbreviated commit hash re-invoked finished deployments, which skipped every episode as already done, and a container restart interrupted the deployments, which resumed with finished episodes kept. One secondary test changed. The registered code tested the round-level contrasts by permuting slots, but a round has one accepted candidate, so each cell's value was copied to its two slots and the permutation split the copies between groups, which is not a valid test. We noticed this after the first run of the analysis, which gave $p = .009$ for first write against no attribution, and replaced the test by an exact sign-flip test over the eight state differences ($p \geq .25$); estimates and intervals are unchanged.

Registered secondary analyses. Not shown in the body. Attribution study: component accuracy, on the 13 failures whose component could be identified from the earlier experiment's repairs, was 4 to 7 of 13 for every method that names one; McNemar's test on exact accuracy for the primary comparison found search right and the judge wrong on 11 τ^2 failures and the reverse on 19 ($p = .20$; pooled over three domains 17 and 24, $p = .35$). Repair experiment: RRSI's analysis against no attribution $+4.0$ points $[-6.4, +14.5]$ ($p = .78$) and the three step groups pooled against no attribution $+3.5$ $[-7.6, +14.6]$ ($p = .20$), both carried by the one candidate whose write gate blocked nearly every write; within domains, the contrasts with RRSI's analysis ranged from $+0.05$ to $+0.52$ points in retail and from -4.3 to $+0.1$ in airline (smallest p of the twelve within-domain contrasts .15); the three step methods named the same step on 0% (airline) and 10% (retail) of failing traces, and any two of them on 10–38%.

Post-hoc analyses. Not registered: the re-scoring under other reference choices, the null-replay rates by step position, the decomposition of the null-free score and the retest agreement among failures both runs found rescuable (Table 7, Figure 2; `attrib/robustness.py`); the split of $R(\hat{k})$ into proposals and their gains, search's first suspect, the null replays' reproduction of the observed action and the split-sample scores (Tables 10, 11; `attrib/review2.py`); the observed-action control (`attrib/orig_control.py`); the second oracle (`attrib/second_oracle.py`, `attrib/groundtruth3.py` with another oracle model, sharing null replays where both oracles flagged a step); the judges with other definitions or without grading information (Table 8; `attrib/rescue_prompt.py`); the gold-deviation rule (Table 9; `attrib/gold_rule.py`); the admissibility screen and the audit sample (Table 12; `attrib/admissibility.py`) and the human audit and the re-scoring without state-changing corrections (`attrib/audit_human.py`, `attrib/audit_rescore.py`); the failures that recover by chance (`attrib/chance_failures.py`); aligned search's exact agreement and cost (`attrib/aligned_search_stats.py`); the screen of the oracle fixes (`attrib/oraclefix_screen.py`); counterfactual search with grading information and the factorial analysis (`attrib/search.py -informed`, `attrib/factorial.py`); in the repair experiment, the rescue score of each group's pointers on its own failures (`attrib/phaseb_accuracy.py`), the oracle-pointer group, run after the registered analysis with held-out results unseen until its deployments had finished (`attrib/phaseb_oracle.py`), the proposers' reading of traces (`attrib/phaseb_reading.py`), the no-attribution group's mean without its worst candidate, the intervals with states grouped by run and the correlation of pointer scores with gains (`attrib/phaseb_review3.py`), the two no-reading groups, run with RRSI's proposer blocked from failing traces and held-out results unseen until their deployments had finished (`attrib/phaseb.py`, `attrib/phaseb_review3.py -noread`); and the checks of Appendix F: accepted-round intervals by run, scores by kind of best rescue step, admissibility by kind, intervention scope, split-half agreement, the oracle without grading and the fresh replays (`attrib/review4.py`, `attrib/review4_runs.py`); the drafts' targets, search with every suspect tested and the full profiles without grading (`attrib/review5.py`, `attrib/search_maxgain.py`, `attrib/blind_full.py`).